Sorted Squared Array

AlgoExpert · Easy · Arrays

The problem

- Given a sorted array (may contain negatives) return the squares in sorted order.
- Two pointers from both ends fill the result from the back.

Intuition

- The largest squares sit at the two ends of a sorted array, because big negatives square to big positives.

Brute force vs optimal

- Brute force: square everything, then sort → O(n log n)
- Optimal: largest squares are at the ends → two pointers, O(n)

Approach

- 1. Put pointers at both ends
- 2. Compare absolute values
- 3. Write the larger square into the result from the back
- 4. Move that pointer inward

Worked example

input



output (filled from the back)



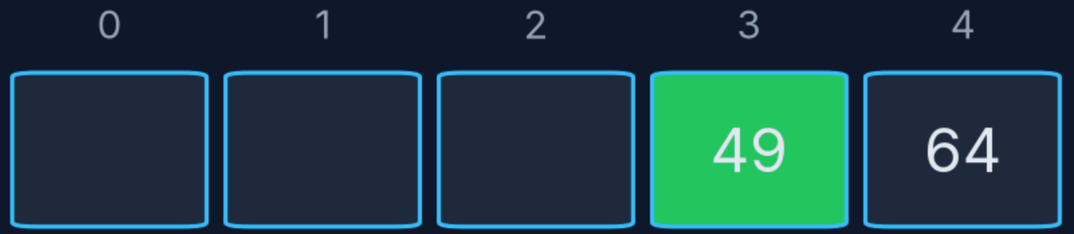
|8| is larger → write 64 at index 4

Worked example

input



output (filled from the back)



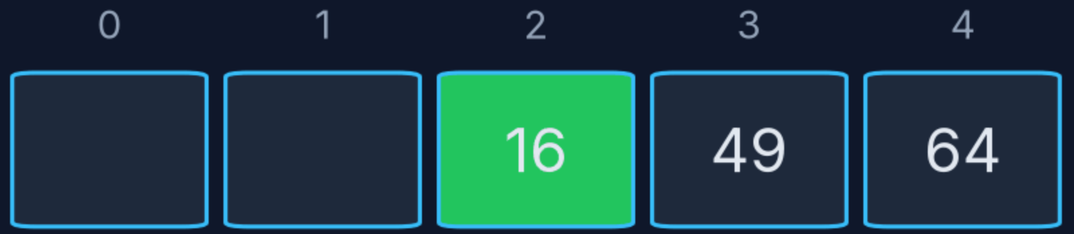
|-7| is larger → write 49 at index 3

Worked example

input



output (filled from the back)



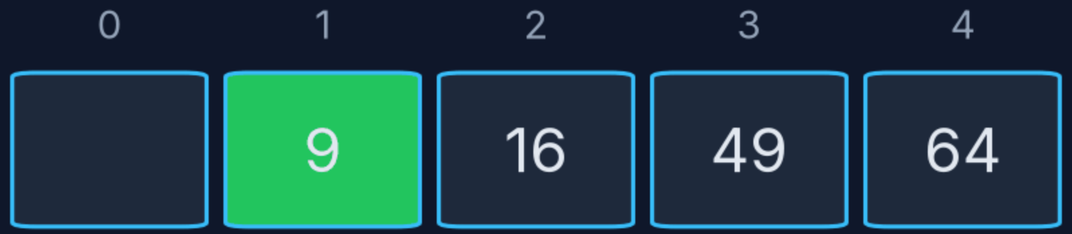
|4| is larger → write 16 at index 2

Worked example

input



output (filled from the back)



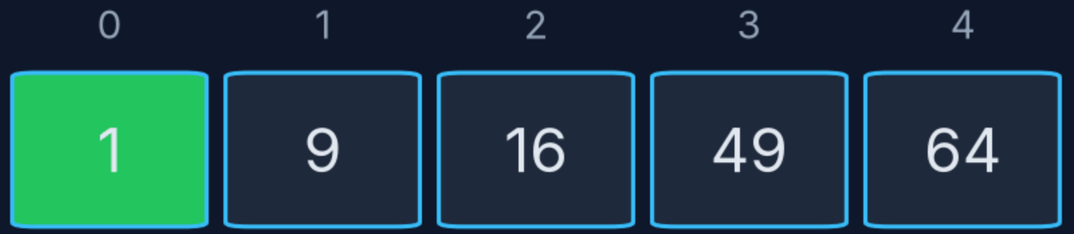
|-3| is larger → write 9 at index 1

Worked example

input



output (filled from the back)



|1| is larger → write 1 at index 0

Java solution

```java
public class SortedSquaredArray {
    public static int[] sortedSquaredArray(int[] array) {
        int[] out = new int[array.length];
        int l = 0, r = array.length - 1;
        for (int k = array.length - 1; k >= 0; k--) {
            if (Math.abs(array[l]) > Math.abs(array[r])) { out[k] = array[l] * array[l]; l++; }
            else { out[k] = array[r] * array[r]; r--; }
        }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- All negatives: squares come out reversed
- Zeros and duplicates are fine
- Fill the output from the back, not the front
- Don't modify the input in place

Interview tips & follow-ups

- State the brute force first, then improve
- Follow-up: do it in place? (not easily — need extra array)
- Generalise: merge of two sorted sequences

Recap

- The largest squares sit at the two ends of a sorted array, because big negatives square to big positives.
- Complexity: O(n)